

Jenkins & GitHub Actions

Senior SDET / Automation Lead • Interview Questions • Practical Pipelines • CI/CD Architecture

1. CI/CD Fundamentals

Q1. What is CI?

Continuous Integration means frequently integrating code changes and automatically building and testing them so defects are detected early.

Q2. What is CD?

Continuous Delivery keeps software releasable through automation. Continuous Deployment automatically deploys approved changes.

Q3. What is the role of Jenkins/GitHub Actions in test automation?

They orchestrate checkout, dependency installation, test execution, reporting, artefacts, notifications and deployment-related stages.

```
Developer push / Pull Request
↓
Checkout source
↓
Build + dependencies
↓
API / Unit tests
↓
UI automation
↓
Reports + screenshots
↓
Artifacts / notification
```

2. Jenkins Core Interview Questions

Q4. What is Jenkins?

An automation server used to orchestrate CI/CD workflows such as builds, tests, packaging and deployments.

Q5. Freestyle vs Pipeline?

Freestyle jobs are configured mainly through the UI. Pipeline-as-code stores workflow logic in a Jenkinsfile, making it versionable and reviewable.

Q6. What is a Jenkinsfile?

A text file containing the Jenkins Pipeline definition, normally committed to source control.

Q7. Declarative vs Scripted Pipeline?

Declarative uses structured sections such as pipeline, agent, stages and post. Scripted provides more Groovy programming flexibility.

Q8. What is an agent?

The machine/executor where Jenkins runs pipeline steps.

Q9. What are stages?

Logical sections such as Checkout, Build, API Tests, UI Tests and Report.

Q10. What is post?

A Declarative Pipeline section used for cleanup, reporting, success and failure handling.

Q11. What are Jenkins credentials?

Securely stored secrets such as tokens, passwords and keys that pipelines consume without hard-coding.

3. Practical Jenkinsfile — Java + Maven + TestNG

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
  }
}
```

```

}

stage('Build') {
  steps {
    sh 'mvn clean install -DskipTests'
  }
}

stage('API Tests') {
  steps {
    sh 'mvn test -Dgroups=api'
  }
}

stage('UI Tests') {
  steps {
    sh 'mvn test -Dgroups=ui'
  }
}

post {
  always {
    // Publish results even after failure.
    junit 'target/surefire-reports/*.xml'
  }

  failure {
    echo 'Pipeline failed.'
  }
}

```

On Windows agents, `bat` is commonly used instead of `sh`.

4. Jenkins Parallel Execution

```

stage('Parallel Automation') {
  parallel {

    stage('Chrome UI') {
      steps {
        sh 'mvn test -Dtest=ChromeTests'
      }
    }

    stage('API') {
      steps {
        sh 'mvn test -Dgroups=api'
      }
    }

    stage('Regression') {
      steps {
        sh 'mvn test -Dgroups=regression'
      }
    }
  }
}

```

Senior point: Pipeline-level parallelism is different from TestNG/Playwright test-level parallelism. Shared test data, environments, browser sessions and reports must remain isolated.

5. Jenkins Scenario Questions

Q12. How do you trigger Jenkins from Git?

Common approaches include SCM webhooks, polling and API-triggered builds. Webhooks are generally preferable when supported.

Q13. How do you pass environment-specific values?

Use parameters, environment variables, credentials and external configuration.

```

parameters {
  choice(
    name: 'ENV',
    choices: ['QA', 'UAT', 'STAGE'],
    description: 'Target environment'
  )
}

```

```
string(  
  name: 'BROWSER',  
  defaultValue: 'chrome',  
  description: 'Browser'  
)  
}
```

Q14. How do you publish reports when tests fail?

Use post/always or equivalent reporting steps. Preserve the failure status while still collecting evidence.

Q15. How do you handle flaky tests?

Identify the root cause first. Use limited retries only for genuinely transient failures; do not hide product defects.

6. GitHub Actions Fundamentals

Q16. What is GitHub Actions?

A GitHub-native automation platform that runs workflows based on repository events, schedules or manual triggers.

Q17. What is a workflow?

A YAML file under .github/workflows/ defining events, jobs, runners and steps.

Q18. What is a job?

A group of steps executed on a runner.

Q19. What is a runner?

The compute environment where a job executes; it can be GitHub-hosted or self-hosted.

Q20. What is a step?

An individual action or shell command inside a job.

Q21. What is an action?

A reusable unit of automation used as a workflow step.

7. Practical GitHub Actions — Playwright + TypeScript

```
name: Playwright Tests  
  
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
  
    steps:  
      # Checkout repository.  
      - name: Checkout  
        uses: actions/checkout@v4  
  
      # Install Node.js.  
      - name: Setup Node  
        uses: actions/setup-node@v4  
        with:  
          node-version: 24  
          cache: npm  
  
      # Install dependencies.  
      - name: Install dependencies  
        run: npm ci  
  
      # Install Playwright browsers.  
      - name: Install Playwright  
        run: npx playwright install --with-deps  
  
      # Run automation.  
      - name: Run tests  
        run: npx playwright test  
  
      # Preserve evidence after failure.  
      - name: Upload report  
        if: always()
```

```
uses: actions/upload-artifact@v4
with:
  name: playwright-report
  path: playwright-report/
```

if: always() is commonly used for diagnostic/report artefacts so test failure does not prevent evidence collection.

8. GitHub Actions Matrix

```
strategy:
matrix:
  browser: [chromium, firefox, webkit]
  environment: [qa, uat]

steps:
  - name: Run tests
    run: npx playwright test
    --project=${{ matrix.browser }}
```

Matrix execution is useful for browser/environment coverage, but the number of job combinations increases execution cost.

9. Jenkins vs GitHub Actions

Area	Jenkins	GitHub Actions
Definition	Automation server	GitHub-native workflow automation
Pipeline	Jenkinsfile	YAML workflow
Execution	Jenkins agents/nodes	Hosted or self-hosted runners
Secrets	Jenkins Credentials	GitHub Secrets/permissions
Triggers	Webhooks, SCM, schedules, manual/API	Push, PR, schedule, manual and GitHub events
Reusable logic	Shared Libraries	Reusable workflows/actions

10. Security Questions

Q22. Should passwords be stored in Jenkinsfile?

No. Use the platform credential/secret mechanism.

Q23. Why avoid printing secrets?

Logs may be visible and retained. Secrets should never be deliberately echoed.

Q24. What is least privilege?

Give workflows only the permissions needed for their tasks.

Q25. How should cloud credentials be handled?

Prefer short-lived/federated credentials where supported instead of long-lived static secrets.

11. Reports & Artifacts

Q26. Test result vs artifact?

A test result communicates pass/fail and metadata. An artifact is a preserved file such as screenshot, video, HTML report, trace or log.

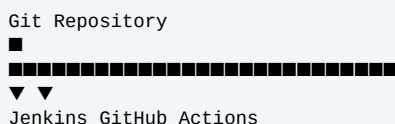
Q27. What should you preserve after UI failure?

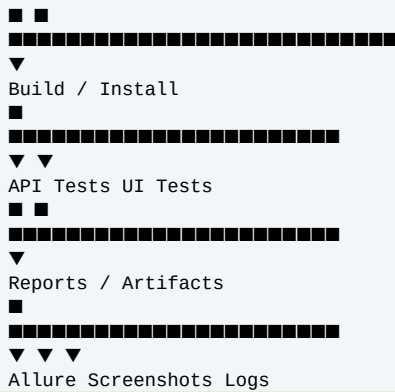
Screenshots, video, traces, browser logs and test reports, depending on framework.

Q28. Should reports upload only on success?

Usually no. Failure evidence is often the most valuable evidence.

12. Senior SDET CI/CD Architecture





Senior answer: “I treat CI as an orchestration layer. The framework owns test logic and reporting; Jenkins or GitHub Actions owns checkout, environment setup, execution, parallelisation, secrets integration, artefact retention and team feedback.”

13. Advanced Interview Questions

Q29. Jenkins agent goes offline?

The build can be interrupted or queued depending on state/configuration. Robust setups use appropriate provisioning, monitoring and infrastructure retry strategies.

Q30. How would you reduce a 2-hour regression?

Measure first; remove unnecessary waits, improve isolation, parallelise independent tests, distribute execution, separate smoke/regression and fix environment bottlenecks.

Q31. How would you design PR validation?

Use fast deterministic checks first: build, unit/API checks and focused UI smoke. Run broader regression after merge or on schedule as appropriate.

Q32. How do you prevent deployment conflicts?

Use environment locks/concurrency controls, queues or environment protection mechanisms.

Q33. What is a reusable GitHub workflow?

A workflow callable by other workflows, allowing common CI logic to be centralised.

Q34. What is a Jenkins Shared Library?

Reusable Jenkins Pipeline code that centralises common pipeline functions and conventions.

14. Scenario-Based Practice

Scenario 1: Tests pass locally but fail in Jenkins.

Check: Java/Node/browser versions, OS, environment variables, credentials, network, headless mode, paths, time zone, test data, resources and parallel execution.

Scenario 2: GitHub Actions takes 45 minutes.

Check: dependency installation, browser installation, test duration, parallelism, matrix size, caching and sequential steps.

Scenario 3: Test fails but report is missing.

Check: artifact/report collection conditions, paths, permissions and retention.

Scenario 4: Secret appears in logs.

Response: revoke/rotate the exposed secret, remove unsafe logging, review access and move the credential to secure CI storage.

15. Rapid Revision — 20 Points

1. Jenkins = automation server.
2. Jenkinsfile = Pipeline as code.
3. GitHub Actions = GitHub-native workflow automation.
4. Workflow = YAML automation definition.
5. Job = execution unit.
6. Step = command/action inside a job.
7. Agent/runner = execution environment.
8. Never hard-code secrets.
9. Publish reports after failures.
10. Preserve screenshots/logs/traces.

11. Use parallel execution carefully.
12. Separate pipeline vs test-level parallelism.
13. Isolate shared test data.
14. PR pipelines should be fast.
15. Regression can run in broader pipelines.
16. Use caching where useful.
17. Measure before optimising.
18. Keep CI orchestration separate from test logic.
19. Jenkins Shared Libraries = reusable Jenkins logic.
20. Reusable GitHub workflows = reusable CI logic.

■ **Senior SDET interview line: “My CI/CD design focuses on fast feedback, deterministic execution, secure secrets, parallelisation without test interference, reliable artefact collection and maintainable pipeline-as-code.”**